

# The Two-Axis AI Engineering Maturity Framework

*A Practitioner's Guide to Assessing and Advancing Human–AI Collaboration in Software Engineering*

---

April 2026

*Includes implementation reference: AgentAlign v1.4.0 by Arpinine*

## Abstract

---

Human–AI collaboration in software engineering is not a single activity. It spans a spectrum from passive autocomplete to fully governed, compounding engineering systems. Yet most organisations assess their position on this spectrum only vaguely — if at all. This paper proposes a clarified, deepened version of the two-axis maturity framework that distinguishes Engineering Process maturity (EP) from Product AI-Nativeness (AIN), and argues that treating these as a single dimension is the most common and costly error in AI engineering strategy.

We deepen each axis with richer indicators, describe the mechanics of each transition, and illustrate progression patterns with illustrative scenarios. Where relevant, we reference AgentAlign — a Claude Code plugin developed by Arpinine — as a concrete implementation of the governance structures the framework prescribes. AgentAlign is not the only way to implement the framework, but it provides a working demonstration of the enforcement model at each EP level.

The central finding is unchanged: the barrier to transformative AI-assisted engineering is not model capability. It is the presence or absence of persistent governance structures, formal contracts between human and AI, and compounding feedback loops that leave each session smarter than the last.

## 1. The Problem With “Using AI”

---

In 2026, “using AI for software development” has become meaningless as a descriptor. A developer accepting tab completions and a domain expert directing multi-application builds within a formal governance framework both “use AI” — yet their productivity, quality, and economic profiles differ by one to two orders of magnitude.

The absence of shared vocabulary creates three practical problems. First, organisations cannot assess where they are, making it impossible to plan credible improvement. Second, investment decisions default to what is visible on procurement dashboards — model licenses and tool subscriptions — rather than what actually drives returns: governance infrastructure. Third, ‘AI-native product’ has become a marketing term that conflates two fundamentally different things: products that have AI features bolted on, and products architecturally designed for AI agents to operate within.

*The most expensive confusion in AI engineering strategy is treating process maturity and product architecture as a single axis. They are independent dimensions that require independent investment — and conflating them produces organisations that are high on one and blind to the other.*

## 2. The Two-Axis Framework

---

The framework measures two dimensions that are independent until the highest levels of maturity, where they converge.

### Axis 1 — Engineering Process Maturity (EP)

EP measures how the human–AI collaboration is governed. It asks: what structures constrain AI behaviour? What contracts define the boundary between human responsibility and AI responsibility? Do sessions compound — does each interaction leave the system measurably smarter than it was before?

EP maturity has nothing to do with which AI model is used. The same model instance produces categorically different results at EP Level 2 versus EP Level 5 — because the governance infrastructure that constrains and focuses it is different.

### Axis 2 — Product AI-Nativeness (AIN)

AIN measures whether the product is designed for AI agency. Crucially, this does not mean ‘does the product have AI features?’ It means: can AI agents compose operations within the product, execute them, observe consequences in real time, and use that feedback to refine subsequent actions?

A product can have sophisticated recommendation engines, generative content, and AI copilots while scoring low on AIN — because none of those features enable AI agents to operate as first-class principals within the system. The distinction is architectural, not cosmetic.

### Why Independence Matters

The axes are independent below their convergence point. This independence reveals two common failure patterns:

- **Pattern A — The capable product, the chaotic process:** High AIN / Low EP: The organisation is building impressive AI-first products, but the engineering process is

ad-hoc. Build velocity is inconsistent, errors recur, and the product's own AI capability outpaces the team's ability to govern its development.

- **Pattern B — The disciplined process, the wrong product:** High EP / Low AIN: The team operates a disciplined, compounding engineering system, but is building products that treat AI as a feature layer rather than an operating principal. The governance infrastructure is excellent; it is being applied to the wrong architectural substrate.

### 3. Engineering Process Maturity — Axis in Depth

---

EP maturity progresses through five meaningful levels. The transition between each level requires a specific investment — not in tooling, but in practice and discipline.

#### EP Level 1 — Ad-Hoc Interaction

The practitioner interacts with AI opportunistically. No project context persists between sessions. No structure governs what the AI receives or what it is expected to produce. The interaction is purely reactive — a question is asked, an answer is received, it may or may not be used.

The ceiling here is low not because the AI is incapable, but because the human is providing insufficient signal. An AI operating without architectural context, persistent constraints, or formal completion criteria will produce plausible-looking output that reflects its training distribution, not the specific requirements of the system being built.

Illustrative scenario: A developer uses an AI chat interface to ask how to implement pagination in FastAPI. They receive a correct general answer, adapt it manually, and move on. The same question will be asked again next month.

#### EP Level 2 — Contextual Direction

The practitioner provides rich context per session: architecture descriptions, API schemas, existing code, constraints. Output quality improves significantly — but the improvement exists only within that conversation. Nothing carries forward. The same errors recur because no mechanism prevents them.

The critical failure mode at this level is mistaking context-dependent quality for systematic quality. The evidence that a practitioner is at EP 2 rather than EP 4 is the recurrence rate of errors across sessions.

Illustrative scenario: A data platform architect provides a detailed Kafka schema and data quality rules before asking the AI to generate a pipeline. The output is good. Two weeks later, a new pipeline task makes the same null-handling error that was caught in the first session — because the lesson was never codified.

#### EP Level 3 — Formal Planning

The practitioner writes implementation plans with explicit completion criteria before the AI begins coding. ‘Done when...’ conditions are stated. Architectural decisions are documented with rationale. Violations are detected during review.

What distinguishes Level 3 from Level 2 is the introduction of verifiable postconditions. The AI is no longer evaluated on ‘does this look right?’ but on ‘does this satisfy the stated criteria?’ This is a significant shift in the governance relationship — it makes the AI’s output falsifiable rather than merely assessable.

What Level 3 lacks is persistence. Lessons are learned, errors are corrected, but the corrections do not propagate. The system does not compound.

► **AgentAlign implementation** (Arpinine — agent-align v1.4.0)

**/spec-new + /spec-review:** Creates spec.md from feature request, then reviews it for measurability and AC format before engineering starts. Each acceptance criterion must match the AC-NNN: pattern — enforcing structured postconditions rather than free-text ‘done’ claims.

**/spec-plan:** Produces plan.md with module boundaries, dependency rules, and testability sections as required gates before implementation. The architecture-readiness hook blocks writes to implementation paths until all three sections are substantively present — making it structurally impossible to skip formal planning.

**check-constitution.sh:** PreToolUse hook that blocks writes to spec.md containing implementation language (30+ keywords). Enforces the boundary between intent and execution at the artefact level.

## EP Level 4 — Systematic Governance (The Compounding Threshold)

Level 4 is the most consequential transition in the framework. The practitioner begins turning corrections into persistent rules — machine-readable constraints that prevent errors from recurring, not just within a project, but across projects and programming languages.

The mechanism is attribution. When the AI makes an error, the practitioner does not simply fix it. They ask: was this a precondition failure (insufficient context, human responsibility) or a postcondition failure (incorrect execution, AI responsibility)? The answer determines the corrective action. Precondition failures require better context provision. Postcondition failures generate new rules.

*The Level 4 threshold is not about capability — it is about discipline. Every ‘done’ claim is verified against the stated postcondition. Every error generates a root cause analysis. Every lesson generates a rule candidate. Every session leaves the system smarter.*

The economic effect of this transition is non-linear. Once rules are persistent, each new project starts at the accuracy ceiling of the accumulated rule set. Errors that consumed hours in early projects consume minutes in later ones — not because the AI improved, but because the governance system matured.

► **AgentAlign implementation** (Arpinine — agent-align v1.4.0)

**/spec-audit (drift attribution):** Classifies every finding as PRECONDITION FAILURE (spec or plan was unclear; refine the artefact), POSTCONDITION FAILURE (spec was explicit; implementation deviated; candidate for a new rule), or RULE VIOLATION (matches an active rule; fix the code). This three-way classification is the operational form of the attribution discipline the framework prescribes.

**rule-manager (add-rule):** When /spec-audit identifies a postcondition failure, it flags it as a rule candidate. /spec-retro extracts lessons from completed work, classifies each as a rule candidate, and persists confirmed rules to .specify/rules/ via rule-manager. Rules are never deleted — only deactivated or superseded — ensuring the compounding property is preserved across projects.

**drift-detector (rule check):** Before creating a new ADR for a drift finding, drift-detector runs rule-manager.check-rules. If the pattern is already captured as a rule, it becomes a RULE VIOLATION requiring code correction — not a new ADR. Known patterns do not regenerate governance overhead.

## EP Level 5 — Operating Model Design

At Level 5, the practitioner designs the governance framework itself as a managed system. Strategic decisions are made about when to sprint versus test, when to deliberately violate rules (with quantified recovery plans), and how to ensure the governance infrastructure survives personnel changes.

The defining test of Level 5 is transferability. A second practitioner entering the system should be able to operate at Level 4 from day one — not because they receive training, but because the rules, plans, lessons, and architectural decisions are machine-readable and comprehensive. If the governance system exists primarily in the originating practitioner's head, the organisation has a Level 5 individual and a Level 2 organisation.

### ► AgentAlign implementation *(Arpinine — agent-align v1.4.0)*

**/spec-status --onboard:** Generates a new team member brief from the current governance artefacts: active rules, open ADRs, eval thresholds, architecture boundaries, and drift status. The brief is derived from machine-readable artefacts, not hand-written documentation — meaning it reflects the actual current state of governance, not the state as of the last time someone updated a wiki.

**Governance table:** /spec-status produces a project-wide governance table covering acceptance criteria status, drift findings, ADR coverage, eval results, and architecture readiness. This is the operational equivalent of the 'build economics tracked at process level' indicator for EP 5 — making the governance system's health visible rather than inferred.

**Artefact schemas:** Machine-readable YAML schemas (spec-schema.yaml, adr-schema.yaml) enforce substance, not just structure. ADRs must include ## Context, ## Decision, ## Consequences. ACs must match AC-NNN: format. This distinguishes substantive governance from compliance theatre — artefacts that pass schema validation are artefacts that would actually inform a new practitioner.

## 4. Product AI-Nativeness — Axis in Depth

AIN maturity describes the degree to which the product’s architecture enables AI agents to operate as first-class principals — not merely as features within the product, but as entities that can compose operations, execute them, observe consequences, and improve subsequent behaviour based on real-time feedback.

## AIN Level 1 — AI-Free or Invisible

The product has no AI involvement in its operation. This is not a criticism — many valuable products are well-suited to what they do. The point is that AIN Level 1 products have no architectural relationship with AI agency, and advancement requires deliberate design, not incremental feature addition.

## AIN Level 2 — AI as Feature Layer

The product incorporates AI capabilities as discrete features: a chatbot, a recommendation engine, a content generation module. These are valuable, but architecturally they are add-ons. The product’s core data model, event architecture, and operational logic were designed for human users and human operators.

Most products described as ‘AI-native’ in 2026 are at this level. The label is applied because AI is visible to end users, not because AI agents have architectural authority within the system.

## AIN Level 3 — AI-Callable Architecture

The product exposes a structured vocabulary — APIs, tool schemas, or domain-specific interfaces — that AI agents can call programmatically. Operations are composable, schemas are documented, and the system can accept AI-initiated commands through defined interfaces.

This is a meaningful architectural shift requiring intentional design investment. Products at AIN Level 3 can be used by AI agents, but the agents cannot observe the consequences of their actions in real time — there is no feedback channel from the product back to the AI.

### ► AgentAlign implementation *(Arpinine — agent-align v1.4.0)*

**AIN Assessment block in spec-schema.yaml:** Specs at AIN  $\geq 3$  must populate `agent_callable_operations` with name, input schema, output shape, and failure modes for each operation. This enforces that the AI-callable vocabulary is explicitly designed before implementation begins — not discovered after the fact.

**Human-in-the-Loop Gates:** The AIN assessment block requires explicit declaration of decisions requiring human approval before an agent proceeds. This makes the human–agent boundary a governed contract rather than an implicit assumption.

**Evaluation Required gate:** Specs with AIN  $\geq 3$  and Evaluation Required: YES block implementation until `eval-plan.md` exists with metrics, thresholds, datasets, and pass/fail policy. AIN Level 3 without an evaluation contract is not a governed system — it is an ungoverned one that happens to have an API.

## AIN Level 4 — Composable with Feedback

The product is designed for AI agents to compose workflows, execute them, observe real-time consequences through event streams or structured feedback channels, and use that feedback to refine subsequent actions. The agent is not calling a black box — it is operating within a system that tells it what happened.

This is the minimum threshold for genuinely AI-native architecture. Below this level, AI agents operating within the product are flying blind: they can initiate actions, but they cannot close the loop.

Illustrative scenario: An operations platform at a mining company exposes a DSL for composing inspection workflows. An AI agent composes a sequence of sensor queries and anomaly checks, executes it, receives a structured event stream indicating which checks passed and which raised alerts, and uses that feedback to adjust the next iteration — without human involvement in each cycle.

► **AgentAlign implementation** (*Arpinine — agent-align v1.4.0*)

**Feedback Channels (AIN assessment block):** The spec template requires explicit declaration of how the system captures signal about agent quality — not as a free-text field, but as a structured section that the evaluation-governor references when determining whether evaluation contracts are sufficient.

**evaluation-governor:** Blocks implementation completion on failing eval thresholds and tracks eval regression. For AIN Level 4 systems, this means the feedback loop the product provides to its agents is itself under governance — regression in feedback quality is detected, not just regression in output quality.

**EP–AIN axis alignment:** A team building an AIN Level 4 product while operating at EP Level 2 is the most dangerous configuration in the framework. AgentAlign’s /spec-status governance table makes this mismatch visible by surfacing EP-level indicators (rule density, ADR coverage, drift attribution counts) alongside AIN-level indicators (eval plan status, feedback channel definition) in a single view.

## AIN Level 5 — Self-Improving System (Frontier)

At the frontier, the feedback loop is causal and recursive. AI agents operating within the product generate experience data that is used to improve the agents themselves — closing the loop between operation and learning at the architectural level. The distinction between ‘building the product’ and ‘operating the product’ begins to dissolve.

This level is aspirational in 2026. The architectural components exist in isolated cases. The governance framework for safe, auditable autonomous operation does not yet exist at production scale.

## 5. Indicators by Axis

The following tables give diagnostic signals for each axis. Early signals describe what the level looks like from the inside when first achieved; mature signals describe it at full operation.

## Engineering Process Maturity — Indicators

| Dimension                     | Early signal                                 | Mature signal                                                                   | Diagnostic question                                                          |
|-------------------------------|----------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>EP 1 — Ad-Hoc</b>          | Occasionally uses AI for code snippets       | No AI involvement in architecture decisions; all errors caught in review        | Do the same errors appear in consecutive sessions?                           |
| <b>EP 2 — Contextual</b>      | Provides file contents before prompting      | Pre-session context ritual; quality collapses when context is thin              | Can you point to a lesson from last month that changed your prompting today? |
| <b>EP 3 — Formal Planning</b> | Writes ‘Done when...’ criteria before coding | All tasks have verifiable gates; violations caught systematically               | Is every AI ‘done’ claim verified against a stated postcondition?            |
| <b>EP 4 — Systematic</b>      | Errors generate rule candidates              | Rule set compounds across projects; human correction time decreasing measurably | Does your rule set prevent errors across projects, or only within them?      |
| <b>EP 5 — Operating Model</b> | Governance tracked as managed system         | A new practitioner reaches EP 4 on day 1 from written artefacts alone           | Would your governance system survive your departure?                         |

## Product AI-Nativeness — Indicators

| Dimension                            | Early signal                               | Mature signal                                                       | Diagnostic question                                                          |
|--------------------------------------|--------------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>AIN 1 — Absent</b>                | No AI in product                           | Deliberately AI-free; no provision for AI agency                    | Is AI excluded by design or by omission?                                     |
| <b>AIN 2 — Feature Layer</b>         | Chatbot or recommendation engine added     | AI visible to users; core architecture designed for human operators | Can an AI agent initiate operations, or only respond to user triggers?       |
| <b>AIN 3 — AI-Callable</b>           | API or tool schema published               | Operations composable by agents; schemas documented                 | Can an agent observe what happened after it took an action?                  |
| <b>AIN 4 — Composable + Feedback</b> | Event stream or feedback channel available | Agents compose, execute, observe, refine in real-time loop          | Does the product tell the AI what happened, or does the AI have to infer it? |

| Dimension                                | Early signal                             | Mature signal                        | Diagnostic question                                   |
|------------------------------------------|------------------------------------------|--------------------------------------|-------------------------------------------------------|
| <b>AIN 5 — Self-Improving (Frontier)</b> | Agent experience feeds model improvement | Build/operate distinction dissolving | Does your product learn from AI-generated operations? |

## 6. Transition Mechanics

Each transition has a specific investment requirement, a common failure mode, and an observable signal that the transition has been made.

| Transition       | EP: from                         | EP: to                                    | AIN: from                        | AIN: to                             |
|------------------|----------------------------------|-------------------------------------------|----------------------------------|-------------------------------------|
| <b>EP 1 → 2</b>  | Opportunistic prompting          | Context-rich prompting as ritual          | No structure                     | Rich pre-session context            |
| <b>EP 2 → 3</b>  | Quality varies by context        | Verifiable postconditions on all tasks    | Quality varies by context        | Quality verifiable against criteria |
| <b>EP 3 → 4</b>  | Corrections fix the session      | Corrections generate persistent rules     | Knowledge in practitioner's head | Knowledge in machine-readable rules |
| <b>EP 4 → 5</b>  | Governance works for originator  | Governance transfers to new practitioners | Individual excellence            | Systemic capability                 |
| <b>AIN 1 → 2</b> | No AI in product                 | AI features visible to users              | Architecture ignores AI          | AI exists as feature layer          |
| <b>AIN 2 → 3</b> | AI features; no agent interface  | AI-callable API or tool schema            | AI as feature consumer           | AI as operation initiator           |
| <b>AIN 3 → 4</b> | Agents can act; cannot observe   | Real-time feedback channel to agents      | One-way command interface        | Closed-loop agent operation         |
| <b>AIN 4 → 5</b> | Feedback informs agent behaviour | Feedback trains agent models              | Operational feedback loop        | Self-improving feedback loop        |

### The Most Consequential Transitions

Two transitions deserve particular attention because they are non-obvious, frequently missed, and disproportionately valuable.

**EP 3 → 4 (The Compounding Threshold).** This is not a capability transition — it is a discipline transition. The test is simple: open your rule files. If they are empty or sparse, you are

at EP 3 regardless of session quality. In AgentAlign terms: if `/spec-retro` has never been run, or if `.specify/rules/` is empty, the system is not compounding.

**AIN 2 → 3 (The Architecture Decision Point).** Products at AIN 2 can reach AIN 3 only through deliberate architectural investment — not incremental feature addition. The question to answer before implementation begins is: what vocabulary of operations should an AI agent be able to compose within this system? AgentAlign enforces this question through the `agent_callable_operations` section of the AIN assessment block, which must be populated before implementation is unblocked for  $AIN \geq 3$  specs.

## 7. The Convergence Zone

At the highest levels of maturity, the two axes converge. When EP reaches Level 5 and AIN reaches Level 4, the governance system that was designed to direct the engineering process becomes applicable to the product's own AI agents. The persistent rules, formal contracts, and compounding feedback loops that govern how an AI coding agent builds the platform also govern how AI agents operate within it.

*At convergence, the distinction between ‘the team that builds the system’ and ‘the system being built’ begins to dissolve. The product is not merely AI-assisted in its construction — it is governed by the same formal structures that governed its construction. The engineering process and the product architecture share the same substrate.*

This convergence has a practical design implication. If the governance infrastructure governing the engineering process is designed with convergence in mind — if the rule format, feedback schema, and contract vocabulary are compatible with the product's own agent interfaces — then the investment in EP maturity compounds into AIN maturity at no additional architectural cost.

### ► AgentAlign implementation (Arpinine — agent-align v1.4.0)

**Convergence by design:** AgentAlign's rule format (`.specify/rules/<category>/<rule-id>.md` with structured YAML frontmatter) and ADR schema (machine-readable `governs:` and `covers:` linkage) are designed to be legible not just to the human practitioners who maintain them, but to AI agents that may eventually operate within the governed system. A platform that exposes its own governance artefacts as agent-callable operations — letting an AI agent query the active rule set, check ADR coverage, or retrieve eval thresholds before taking an action — is a system that has achieved convergence.

**The `/spec-status` governance table as convergence signal:** When a team can look at `/spec-status` and see both EP-level indicators (rule density, attribution counts, onboarding readiness) and AIN-level indicators (eval plan status, feedback channel definition, agent-callable operations) in a single view, and when those two sets of indicators are trending together rather than diverging, the system is approaching convergence. When they diverge — high AIN ambition, low EP discipline — the governance table makes the misalignment visible before it becomes expensive.

## 8. Self-Assessment Checklist

---

Answer each question honestly. Your position is the highest level at which you can answer yes to all questions in that group.

### Engineering Process Maturity

- Do you provide rich architectural context (schemas, constraints, existing code) before each AI task? (EP 2+)
- Do you write explicit ‘Done when...’ criteria before the AI begins implementation? (EP 3+)
- Is every AI ‘done’ claim verified against the stated postcondition? (EP 3+)
- When the AI makes an error, do you attribute it to precondition or postcondition failure — and generate a persistent rule accordingly? (EP 4+)
- Is each new project measurably cheaper and more accurate than the last, due to compounded rules? (EP 4+)
- Could a new practitioner inherit your governance system and operate at EP 4 from day one? (EP 5+)
- Do you track build economics at the process level, not just per-tool license cost? (EP 5+)

### Product AI-Nativeness

- Does your product expose a structured vocabulary that AI agents can call programmatically? (AIN 3+)
- Are the operations composable — can an agent construct multi-step workflows from them? (AIN 3+)
- Does your product provide a feedback channel that tells an agent what happened after it acted? (AIN 4+)
- Can an agent close the loop — compose, execute, observe, refine — without human involvement in each cycle? (AIN 4+)
- Does agent-generated operational data feed back into model or agent improvement? (AIN 5 — frontier)

## 9. Investment Implications

---

The two-axis framework clarifies why most organisations’ AI engineering investments produce marginal returns. The dominant procurement pattern — better models, more tool licenses, prompt engineering training — optimises for EP Level 1–2. It makes individual sessions slightly better. It does not produce compounding returns.

The investments that produce transformative returns are less visible on procurement dashboards: time invested in building persistent rule sets, formal planning infrastructure,

attribution discipline, and governance transferability. These investments take weeks to accumulate — but they require sustained practice, not tool purchases.

On the product side, the AIN 2'3 transition requires a deliberate architectural decision most cheaply made before the product is built. Organisations undergoing significant platform renovation have a narrow window in which this transition is affordable. After that window closes, the cost is architectural disruption rather than architectural investment.

*The most important investment question is not ‘which AI model should we use?’ It is ‘what governance infrastructure do we need to build, and which architectural decisions must we make now to enable convergence later?’*

## 10. Limitations and Honest Caveats

---

This framework is a practitioner synthesis, not an empirical study. The following limitations apply.

- The EP axis is grounded in measured production results, but from a narrow evidence base. Compounding curve data is from individual practitioners, not multi-team organisations.
- The AIN axis above Level 3 is partially aspirational. Level 4 architectures exist; Level 5 governance for production autonomous operation does not yet exist at scale.
- Domain expertise is not separable from EP maturity in available evidence. The framework cannot currently isolate how much of reported productivity returns is governance infrastructure versus tacit expertise.
- The convergence thesis is a design proposition, not yet a demonstrated case at scale.
- Regarding AgentAlign specifically: the rule trigger evaluation mechanism (natural-language trigger conditions evaluated against file content) introduces potential inconsistency in a layer that should be deterministic. A rule system that fires inconsistently does not compound — practitioners learn to treat violations as advisory rather than definitive. This is a known open problem in the v1.4.0 implementation.

## Conclusion

---

The two-axis framework provides vocabulary that the AI engineering conversation currently lacks. Separating Engineering Process maturity from Product AI-Nativeness reveals a specific error that most organisations are making, and points to the specific investments that would correct it.

The barrier to transformative AI-assisted engineering is not model capability. It is the presence or absence of persistent governance structures: formal contracts between human and AI, machine-readable rules that compound across sessions, and verification disciplines that make AI output falsifiable rather than merely assessable.

AgentAlign, developed by Arpinine, represents one concrete implementation of the governance structures this framework prescribes — from the formal planning hooks of EP Level 3, through the attribution and rule-compounding machinery of EP Level 4, to the transferability artefacts of EP Level 5, and the AIN assessment contracts that govern AI-callable product design. It is not the only implementation of the framework, but it is the most fully realised one currently available, and it demonstrates that the governance structures described here are not merely theoretical.

We offer both the framework and the implementation reference as starting points. Both will be refined by evidence from multiple organisations, multiple domains, and multiple AI platforms. We invite practitioners to apply the checklist, identify their position on both axes, and report whether the transition mechanics described here match their experience.

---

*AgentAlign is available at: <https://arpinine.com/agent-align>*

*© 2026. This white paper may be reproduced for non-commercial use with attribution.*